Qualité Logicielle

Vérification déductive de programmes - Part 2

Asma BERRIRI

asma.berriri@universite-paris-saclay.fr

Page web du cours :
https://sites.google.com/view/asmaberriri/home/app5modelisation-verification

Plus faible précondition - Weakest liberal precondition

- Rappel: Comment décider si un triplet de Hoare est valide ?
  - Se référer à la sémantique opérationnelle du langage : un peu difficile
  - Utiliser un système de déduction pour démontrer qu'un triplet est correct : Logique de Hoare (chapitre précédent)
  - Appliquer un algorithme qui permet de tester qu'un triplet de Hoare est valide : calcul de la plus faible précondition (weakest precondition)

Programmes annotés

But

Automatiser les preuves en logique de Hoare

Nous augmentons notre langage simple avec des invariants de boucle explicites

$c ::= \mid$	$skip$	sans effet
$\mid$	$x := e$	affectation
$\mid$	$c \; ; \; c$	séquence
$\mid$	if b then c else c	conditionnelle
$\mid$	while b *invariant Inv* do c	boucle annotée

La sémantique opérationnelle est inchangée

Weakest Liberal Precondition [Dijkstra 1975]

Définition (WLP ou WP)

- Soit c un programme et Q une formule (assertion)
- On notera $WP(c, Q)$ la plus faible précondition telle que si c termine, alors c termine dans un état qui satisfait Q
  - C'est une précondition qui amène à Q : $\{WP(c, Q)\}c\{Q\}$ est valide
  - C'est la plus faible : toute précondition qui amène à Q est plus forte (elle implique donc la plus faible)
  - Si P et Q sont des assertions alors P est 'plus faible' que Q si $Q \implies P$
- Pour tout prédicat P, si $\{P\}c\{Q\}$ alors $P \implies WP(c, Q)$

Théorème (correction partielle avec WP)

Pour prouver $\{P\}c\{Q\}$ il suffit de prouver $P \implies WP(c, Q)$

Calcul de la plus faible précondition WP

- Partez de la postcondition et revenez en arrière
- Travaillez à rebours, en calculant les préconditions les plus faibles, une instruction à la fois, pour en dériver la précondition la plus faible pour le programme dans son ensemble

```
stmt1                {c0} stmt2 {c1}
stmt2                {c1} stmt2 {c2}
...       ---->
stmtn                {cn-1} stmtn {cn}
```

↪ cn = postcondition

↪ $c0$ = weakest precondtion

Precondition $\Longrightarrow$ $c0$? ?

- Plus formellement, on définit une fonction *WP* qui prend 2 arguments: $WP(c, Q)$
  - ↪ c: un programme/procédure/fonction ou un fragment de celui-ci,
  - ↪ Q: une formule qui est la postcondition du c
  - ↪ Doit renvoyer la précondition minimale P qui valide le triplet $\{P\}\ c\ \{Q\}$

⇒ *WP* est défini *par cas* sur les instructions

WP : le cas de l'instruction vide *skip*

- $\{Q\}skip\{Q\}$ est un triplet valide, pour tout prédicat Q
- *skip* ne change pas les états

$$\boxed{WP(skip, Q) = Q}$$

WP : le cas de l'affectation

$$\boxed{WP(x := e, Q) = Q[x \leftarrow e]}$$

- ↪ $Q[x \leftarrow e]$: c'est la formule Q où toutes les occurrences libres[1] de x sont remplacées par e
  e.g. Si Q est $x > 1$, alors $Q[x \leftarrow x + 1]$ est $x + 1 > 1$
- ↪ Les affectations changent l'état, nous nous attendons donc à ce que les triples Hoare pour les affectations reflètent toujours ce changement

- Exemple:
  $WP(x := x + 1, x > 5) = (x + 1) > 5 = x > 4$
  $x > 4 \implies x > 4$
  $\{x > 4\} x := x + 1 \{x > 5\}$ est un triplet de Hoare valide
- Exercice
  Trouver la précondition la plus faible P

$$\{P\}\ x = y + 1\ \{x = 3\}$$

[1] Soient x une variable et A une formule. Dans une formule $\forall x\ A$ ou $\exists\ xA$, la portée de la liaison pour x est A. Une occurrence de x dans une formule est libre si elle n'est pas dans la portée d'une liaison pour x, sinon elle est dite liée.

WP : le cas de la séquence

$$\boxed{WP(c1; c2, Q) = WP(c1, WP(c2, Q))}$$

- Calculer la précondition la plus faible WP pour $c2$, l'utiliser comme postcondition pour $c1$
  - ↪ On considère la séquence $c1; c2$ avec la postcondition Q
  - ↪ Nous travaillons à rebours (backward)
  - ↪ Nous calculons d'abord $WP(c2, Q)$. Appelons P le prédicat $WP(c2, Q)$
  - ↪ Ensuite nous calculons $WP(c1, P)$
  - ↪ Le résultat est la précondition la plus faible pour la séquence $c1; c2$ par rapport à la postcondition Q
- Exemple :

$$\begin{aligned}
&WP(x := x + 1; x := x + 2, x > 5)\\
=&WP(x := x + 1, WP(x := x + 2; x > 5))\\
=&WP(x := x + 1, (x + 2) > 5) = WP(x := x + 1, x > 3)\\
=&(x + 1) > 3 = x > 2
\end{aligned}$$

- Même méthode pour une séquence de 3 instructions, 4 instructions etc.

WP : Séquence- Exercice

Calculez la plus faible précondition pour la séquence d'instructions d'affectation suivante $a := 2 * b + 1; b := a - 3$, pour la postcondition $b < 0$.
$WP(a := 2 * b + 1; b := a - 3, b < 0) = ?$

$$
\begin{aligned}
& WP(a := 2 * b + 1; b := a - 3, b < 0) \\
& = WP(a := 2 * b + 1, WP(b := a - 3, b < 0)) \\
& = WP(a := 2 * b + 1, (b < 0)[b \leftarrow a - 3]) \\
& = WP(a := 2 * b + 1, (a - 3 < 0)) \\
& = WP(a := 2 * b + 1, a < 3) \\
& = (a < 3)[a \leftarrow 2 * b + 1] \\
& = 2 * b + 1 < 3 \\
& = b < 1
\end{aligned}
$$

Donc $\{b < 1\} a := 2 * b + 1; b := a - 3 \{b < 0\}$ est un triplet de Hoare valide

WP : Séquence- Exercices

- Exercice-1: Soit *Prog* le programme ci-dessous. Nous voulons le prouver correct par rapport au contrat

```
/*@
Requires x = a ∧ y = b
Ensures x = b ∧ y = a
*/
temp := x;
x:= y;
y := temp
```

- Que fait *Prog*?
- Prouver que $\{x = a \wedge y = b\} Prog \{x = b \wedge y = a\}$ est valide
  1. Calculons d'abord $W = WP(Prog, x = b \wedge y = a)$, la plus faible précondition
  2. Ensuite, prouvons que $(x = a \wedge y = b) \implies W$
- Exercice-2: Trouver la précondition la plus faible P

$$\{P\}\ x = x - 2; z = x + 1\ \{z \neq 0\}$$
$$\{P\}\ x = 2 * y; z = x + y\ \{z > 0\}$$
$$\{P\}\ w = 2 * w; z = -w; y = v + 1; x = min(y, z)\ \{x < 0\}$$

WP : Séquence Solution Exo2 (1)

$$
\begin{aligned}
P &= WP(x = x - 2; z = x + 1, z \neq 0) \\
&= WP(x = x - 2, WP(z = x + 1, z \neq 0)) \\
&= WP(x = x - 2, (z \neq 0)[z \leftarrow x + 1]) \\
&= WP(x = x - 2, (x \neq -1)) \\
&= (x \neq -1)[x \leftarrow x - 2] \\
&= x - 2 \neq -1 = \ x \neq 1
\end{aligned}
$$

$$\{P\} = \{x \neq 1\}$$

$$
\begin{aligned}
P &= WP(x = 2 * y; z = x + y, z > 0) \\
&= WP(x = 2 * y, WP(z = x + y, z > 0)) \\
&= WP(x = 2 * y, (z > 0)[z \leftarrow x + y]) \\
&= WP(x = 2 * y, (x + y > 0)) \\
&= (x + y > 0)[x \leftarrow 2 * y] \\
&= (2 * y + y) > 0 \ = \ (3 * y > 0) \ = \ (y > 0)
\end{aligned}
$$

$$\{P\} = \{y > 0\}$$

WP : Séquence Solution Exo2 (2)

3. Trouver la précondition la plus faible

$$\{P\}\ w = 2 * w; z = -w; y = v + 1; x = min(y, z)\ \{x < 0\}$$

$$\begin{aligned}
P &= WP(w = 2 * w; z = -w; y = v + 1; x = min(y, z), x < 0) \\
&= WP(w = 2 * w, WP(z = -w; y = v + 1; x = min(y, z), x < 0)) \\
&= WP(w = 2 * w, WP(z = -w, WP(y = v + 1; x = min(y, z), x < 0))) \\
&= WP(w = 2 * w, WP(z = -w, WP(y = v + 1, WP(x = min(y, z), x < 0)))) \\
&= WP(w = 2 * w, WP(z = -w, WP(y = v + 1, (x < 0)[x \leftarrow min(y, z)]))) \\
&= WP(w = 2 * w, WP(z = -w, WP(y = v + 1, (min(y, z) < 0)))) \\
&= WP(w = 2 * w, WP(z = -w, WP(y = v + 1, (y < 0 \vee z < 0)))) \\
&= WP(w = 2 * w, WP(z = -w, (y < 0 \vee z < 0)[y \leftarrow v + 1])) \\
&= WP(w = 2 * w, WP(z = -w, (v + 1 < 0 \vee z < 0))) \\
&= WP(w = 2 * w, (v + 1 < 0 \vee z < 0)[z \leftarrow -w]) \\
&= WP(w = 2 * w, (v + 1 < 0 \vee -w < 0)) \\
&= (v + 1 < 0 \vee -w < 0)[w \leftarrow 2 * w] \\
&= (v + 1 < 0 \vee -2 * w < 0) \\
\{P\} &= \{v < -1 \vee w > 0\}
\end{aligned}$$

WP : le cas des instructions conditionnelles

$$\boxed{\begin{array}{l} WP(IF\ b\ THEN\ c1\ ELSE\ c2\ ENDIF, Q) \\ = (b \implies WP(c1, Q)) \wedge (\neg b \implies WP(c2, Q)) \end{array}}$$

- Lorsqu'une condition est exécutée, $c1$ ou $c2$ est exécuté
- Par conséquent, si le conditionnel doit établir Q, $c1$ et $c2$ doivent tous deux établir Q
- Le choix entre $c1$ et $c2$ dépend de l'évaluation de b dans l'état initial, nous pouvons donc également supposer que b est une précndition pour $c1$ et $\neg b$ est une précondition pour $c2$
- Ainsi, b doit impliquer la précondition la plus faible de $c1$ pour la postcondition Q. Et $\neg b$ doit impliquer la précondition la plus faible de $c2$ pour la postcondition Q

WP : conditionnel: Exemple

- Calculez la plus faible precondition de ce *Pg*

$$IF\ x > 2\ THEN\ y := 1\ ELSE\ y := -1\ ENDIF$$

pour la postcondition $y > 0$

$$\begin{aligned}
&WP(\text{IF } x > 2 \text{ THEN } y := 1 \text{ ELSE } y := -1 \text{ ENDIF}, y > 0) = \\
&((x > 2) \implies WP(y := 1, y > 0)) \wedge (\neg(x > 2) \implies WP(y := -1, y > 0)) = \\
&((x > 2) \implies (1 > 0)) \wedge (\neg(x > 2) \implies (-1 > 0)) = \\
&((x > 2) \implies true) \wedge ((x \leq 2) \implies false) = \\
&(x > 2)
\end{aligned}$$

si vous n'êtes pas convaincu par la dernière étape !!

$$\begin{aligned}
&(x > 2 \implies true) \wedge (x \leq 2 \implies false) \iff \\
&(\neg(x > 2) \vee true) \wedge (\neg(x \leq 2) \vee false) \iff \\
&\neg(x \leq 2) \iff x > 2
\end{aligned}$$

Ainsi si on exécute *Pg* dans un état où x a une valeur > 2 à la fin on est assuré d'obtenir un y dont la valeur est > 0

WP : conditionnel: Exemple

- Que pensez-vous de ce programme ?

```
/*@ requires x > 0
    ensures y > 0
@*/
IF x > 2 THEN y := 1 ELSE y := -1 ENDIF
```

- Comme calculé précédemment, la précondition la plus faible WP est $x > 2$
- Mais la précondition $x > 0$ n'implique pas la précondition la plus faible $x > 2$
- Le programme n'est donc pas correct par rapport à sa spécification

WP : Conditionnel Exercice

Trouver la précondition la plus faible P

$$\{P\}\ \textit{if}\ (x > 0)\ \textit{then}\ y := z;\ \textit{else}\ y := -z\ \{y > 5\}$$

$$\{P\}\ \textit{if}\ (x \neq 0)\ \textit{then}\ z := x;\ \textit{else}\ z := x + 1\ \{z > 0\}$$

WP : Conditionnel Solution (1)

$$\{P\}\ \textit{if}\ (x > 0)\ \textit{then}\ y := z;\ \textit{else}\ y := -z\ \{y > 5\}$$

$$\begin{aligned}
P &= WP(\textit{if}\ (x > 0)\ \textit{then}\ y := z;\ \textit{else}\ y := -z, y > 5) \\
&= ((x > 0) \implies WP(y := z, y > 5)) \wedge (\neg(x > 0) \implies WP(y := -z, y > 5)) \\
&= ((x > 0) \implies (y > 5)[y \leftarrow z]) \wedge ((x \leq 0) \implies (y > 5)[y \leftarrow -z]) \\
&= ((x > 0) \implies (z > 5)) \wedge ((x \leq 0) \implies (-z > 5)) \\
&= ((x > 0) \implies (z > 5)) \wedge ((x \leq 0) \implies (z < -5)) \\
&= ((x \leq 0) \vee (z > 5)) \wedge ((x > 0) \vee (z < -5)) \\
&= ((x \leq 0) \wedge (x > 0)) \vee ((x \leq 0) \wedge (z < -5)) \vee ((z > 5) \wedge (x > 0)) \vee ((z > 5) \wedge (z < -5)) \\
&= (\textit{false}) \vee ((x \leq 0) \wedge (z < -5)) \vee ((z > 5) \wedge (x > 0)) \vee (\textit{false}) \\
&= ((x \leq 0) \wedge (z < -5)) \vee ((z > 5) \wedge (x > 0))
\end{aligned}$$

$$\{P\} = \{((x \leq 0) \wedge (z < -5)) \vee ((z > 5) \wedge (x > 0))\}$$

WP : Conditionnel Solution (2)

$$\{P\}\ \mathit{if}\ (x \neq 0)\ z := x;\ \mathit{else}\ z := x + 1\ \{z > 0\}$$

$$\begin{aligned}
P &= WP(\mathit{if}\ (x \neq 0)\ z := x;\ \mathit{else}\ z := x + 1, z > 0) \\
&= ((x \neq 0) \implies WP(z := x, z > 0)) \wedge (\neg(x \neq 0) \implies WP(z := x + 1, z > 0)) \\
&= ((x \neq 0) \implies (z > 0)[z \leftarrow x]) \wedge ((x = 0) \implies (z > 0)[z \leftarrow x + 1]) \\
&= ((x \neq 0) \implies (x > 0)) \wedge ((x = 0) \implies (x > -1)) \\
&= ((x = 0) \vee (x > 0)) \wedge ((x \neq 0) \vee (x > -1)) \\
&= ((x = 0) \wedge (x \neq 0)) \vee ((x = 0) \wedge (x > -1)) \vee ((x > 0) \wedge (x \neq 0)) \vee ((x > 0) \wedge (x > -1)) \\
&= (x = 0) \vee (x > 0) \vee (x > 0) \\
&= (x \geq 0)
\end{aligned}$$

$$\{P\} = \{x \geq 0\}$$

WP : Conditionnel Exercice

calculez:

$$WP(\text{IF } b \text{ THEN } c \text{ ENDIF}, Q) = ?$$

IF b THEN c ENDIF

est la même instruction que

IF b THEN c ELSE *skip* ENDIF

$WP(\text{IF } b \text{ THEN } c \text{ ENDIF}, Q) =$

$WP(\text{IF } b \text{ THEN } c \text{ ELSE } skip \text{ ENDIF}, Q) =$

$$((b) \implies WP(c, Q)) \wedge (\neg(b) \implies WP(skip, Q)) =$$
$$((b) \implies WP(c, Q)) \wedge (\neg(b) \implies (Q))$$

WP : Conditionnel Exercice 2

Prouvez que ce programme

$$\text{IF } x < y \text{ THEN } m := y \text{ ELSE } m := x \text{ ENDIF}$$

calcule dans m le maximum de x et y?

1. Donnez la postcondition du programme et la précondition le cas échéant
2. Calculez la plus faible precondition *WP*
3. Conclure quand à la correction (validité) de ce programme

WP : Conditionnel Exercice 2

Prouvez que IF $x < y$ THEN $m := y$ ELSE $m := x$ ENDIF calcule dans m le maximum de x et y

1. Aucune precondition (ou la precondition est égale à vrai)

   Postcondition : $m \geq x \wedge m \geq y \wedge (m = x \vee m = y)$

2. the weakest precondition

$$
\begin{array}{l}
\text{IF}(x < y) \\
\quad \text{THEN } \{y \geq x \wedge y \geq y \wedge (y = x \vee y = y)\} \\
\qquad m := y \\
\qquad \{m \geq x \wedge m \geq y \wedge (m = x \vee m = y)\} \\
\quad \text{ELSE } \{x \geq x \wedge x \geq y \wedge (x = x \vee x = y)\} \\
\qquad m := x \\
\qquad \{m \geq x \wedge m \geq y \wedge (m = x \vee m = y)\} \\
\quad \text{ENDIF } \{m \geq x \wedge m \geq y \wedge (m = x \vee m = y)\}
\end{array}
$$

   Ainsi (après simplifications) $WP = (x < y \implies y \geq x) \wedge (x \geq y \implies x \geq y)$ qui est une formule valide

3. Conclude. La précondition (vrai) implique la précondition la plus faible $true \implies (x < y \implies y \geq x) \wedge (x \geq y \implies x \geq y)$

WP : Le cas de la boucle

Considérons le triplet

$$\{P\}\ \textit{while}\ B\ \textit{do}\ c\ \{Q\}$$

Si I est l'invariant, alors il doit être vrai que :

1. si P est vrai alors I doit être vrai, donc $P \implies I$
2. pour chaque itération de c :
   - B est vrai en début de boucle, une précond.
   - I est vrai au début et à la fin, une postcond.

   on a donc le triplet $\{I \wedge B\}\ c\ \{I\}$
3. après la fin de la boucle, I est vrai et B est faux
   - ensemble ils doivent impliquer la post-condition Q
   - $I \wedge \neg B \implies Q$

P doit établir I avant la boucle

I doit être maintenu dans la boucle

conclure la boucle avec la postcondition Q

Deviner les invariants de boucle

Quelques astuces :

- $\hookrightarrow$ A généralement la même forme que la postcondition
- $\hookrightarrow$ Mais dépend d'une manière ou d'une autre de l'index de boucle j
- $\hookrightarrow$ Négation de la condition de boucle $\wedge$ invariant $\implies$ postcondition
- $\hookrightarrow$ Bonne idée : remplacez N par j dans la postcondition

Correction partielle/correction totale

- Prouver $\{P\}\ c\ \{Q\}$ ne suffit pas car il n'y a aucune garantie que la boucle se termine. C'est partiel.

Rappel:

Correction partielle

Montrer que si la boucle c est exécutée sur un état satisfaisant P, si cela se termine, ce sera dans un état satisfaisant Q

Correction totale

Montrer que si la boucle c est exécutée sur un état satisfaisant P, cela finira dans un état satisfaisant Q. En d'autres termes, nous devons prouver la correction partielle et également prouver que la boucle se termine

Correction partielle/correction totale

Pour prouver la correction totale d'un programme, il faut

- prouver sa correction partielle,
- s'assurer que les boucles (et les fonctions récursives, ce que nous ne considérons pas dans ce cours) s'arrêtent nécessairement lorsque les pré-conditions sont vérifiées: la règle de loop devient la suivante, en exhibant une expression, le variant, qui doit :
  - Être une fonction des variables du programme
  - Être positive lorsqu'on entre dans la boucle (c'est-à-dire lorsque la condition de la boucle est vérifiée et lorsque l'invariant de la boucle est vérifié)
  - Décroître strictement à chaque itération de la boucle (en restant positif)

$$\frac{\forall\, V, [P \wedge B \wedge v = V]\ c\ [P \wedge v < V \wedge v \geq 0]}{[P]\ \textit{while}\ B\ \textit{do}\ c\ [P \wedge \neg B]}$$

Correction partielle/correction totale

- Correction partielle $\{P\}$ *while B do c* $\{Q\}$
  - Trouver un invariant *Inv* tel que:
    - $P \implies Inv$ (*Inv* initialement true)
    - $\{Inv \wedge B\}\ c\ \{Inv\}$ (chaque execution du corps de la loupe maintient la validité de *Ind*)
    - $Inv \wedge \neg B \implies Q$ (*Inv* et la condition d'arrêt de la boucle implique la postcondition)
- Correction totale (la boucle se termine) $[P]$ *while B do c* $[Q]$
  - Trouver une fonction variant v telle que
    - $(Inv \wedge B) \implies v > 0$ (v est évaluée à une valeur entière finie supérieure à zéro au début de la boucle)
    - $\{Inv \wedge B \wedge v = V\} c \{v < V \wedge v \geq 0\}$ (La valeur de la fonction variante diminue à chaque fois que le corps de la boucle exécute (ici V est une constante))

WP : Boucle Exercice

Soit le programme *Prog* suivant, avec $N \in \mathbb{N}$

$\{N \geq 0\}$

```
j := 0;
s := 0;
while (j < N) do
    j := j + 1;
    s := s + a[j];
done
```

$\{s = (\sum_i a[i] \mid 0 \leq i < N)\}$

1. Qu'est ce que *Prog* est supposé faire?
2. Prouver, en utilisant Hoare, que *Prog* est incorrecte!

WP : Boucle Exercice

$\{N \geq 0\}$

Prog

```
j := 0;
s := 0;
while (j < N) do
    j := j + 1;
    s := s + a[j];
done
```

$\{s = (\sum_i a[i] \mid 0 \leq i < N)\}$

1. *Prog* calcule dans s la somme des éléments d'un tableau a
2. Comment trouver un invariant *Inv*?
   - ↪ Remplacer N par j dans la postcondition
   - ↪ Ajouter des informations sur la plage de j

   - ↪ Résultat : $Inv = 0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j)$

WP : Boucle Exercice: Que devrions nous prouver?

- Étant donné la précondition, $P = N \geq 0$
- Nous voudrions prouver:
  1. Etablir *Inv*: **(1)** $P \implies Inv$, autrement dit, prouver

     $$\{P\}\ j := 0; s := 0;\ \{Inv\}$$

  2. Maintenir *Inv*: **(2)**

     $$\{Inv \wedge B\}\ c\ \{Inv\} \iff$$
     $$\{Inv \wedge B\}\ j := j + 1; s := s + a[j]\ \{Inv\}$$

  3. Conclure: **(3)**

     $$Inv \wedge \neg B \implies Q$$

- Après la boucle, le résultat que nous voulons Q est

$$s = (\sum_i a[i] \mid 0 \leq i < N)$$

Boucle Exercice: Prouvons le (1)

$$\{P\}\ j := 0; s := 0\ \{Inv\} \iff \{N \geq 0\}\ j := 0; s := 0\ \{0 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j)\}$$

$$\begin{aligned}
W &= WP(j := 0; s := 0,\ 0 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j)) \\
&= WP(j := 0, WP(s := 0,\ 0 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j))) \\
&= WP(j := 0,\ (0 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j))[s \leftarrow 0]) \\
&= WP(j := 0,\ (0 \leq j \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < j))) \\
&= (0 \leq j \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < j))[j \leftarrow 0] \\
&= 0 \leq 0 \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < 0)
\end{aligned}$$

Boucle Exercice: Prouvons le **(1)** cond

$P \implies W$?

$N \geq 0 \implies 0 \leq 0 \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < 0)$?

$N \geq 0 \implies 0 \leq 0 \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < 0)$

$= (N \geq 0) \implies (0 \leq N \wedge 0 = 0)$ ($0 \leq 0$ true, somme vide est 0)

$= (N \geq 0) \implies (0 \leq N)$ ($0 = 0$ true, $P \wedge P$ est true)

$= true$

Donc l'invariant est établi

Boucle Exercice: Prouvons le (2)

$$\{0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \wedge j < N\} \ j := j+1; s := s + a[j]$$

$$\{0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j)\}$$

$$W = WP(j := j+1; s := s + a[j], 0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j))$$

$$= WP(j := j+1, (0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j))[s \leftarrow s + a[j]])$$

$$= WP(j := j+1, (0 \leq j \leq N \wedge \ s + a[j] = (\sum_i a[i] \mid 0 \leq i < j)))$$

$$= (0 \leq j \leq N \wedge \ s + a[j] = (\sum_i a[i] \mid 0 \leq i < j))[j \leftarrow j+1]$$

$$= (0 \leq j+1 \leq N \wedge \ s + a[j+1] = (\sum_i a[i] \ \mid \ 0 \leq i < j+1))$$

Boucle Exercice: Prouvons le **(2)** cond

$$0 \leq j < N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \implies w \ ?$$

$$0 \leq j < N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \implies$$

$$(-1 \leq j \leq N \wedge \ s + a[j+1] = (\sum_i a[i] \mid 0 \leq i < j+1))$$

(simplifier les bornes de j)

$$= (0 \leq j < N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j)) \implies$$

$$(-1 \leq j \leq N \wedge \ s + a[j+1] = (\sum_i a[i] \mid 0 \leq i < j) + a[j])$$

(séparer le dernier terme de la somme)

Problème: il faut que $a[j+1]$ et $a[j]$ s'annulent

Boucle Exercice: Où est l'erreur ?

Prog incorrect:

$\{N \geq 0\}$

```
j := 0;
s := 0;
while (j < N) do
Il faut ajouter l'élément à s
     avant d'incrémenter j
     j := j + 1;
     s := s + a[j];
done
```

$\{s = (\sum_i a[i] \mid 0 \leq i < N)\}$

*Prog*1 correct donc:

$\{N \geq 0\}$

```
j := 0;
s := 0;
while (j < N) do
     s := s + a[j];
     j := j + 1;
done
```

$\{s = (\sum_i a[i] \mid 0 \leq i < N)\}$

- Refaites la preuve pour démontrer *Prog*1 svp :)
- Trouver un variant, et prouver la terminaison de *Prog*1
- Conclure quand à la correction totale de *Prog*1

Correction: Obligations de Preuve (PO) pour *Prog*1

- Étant donné la précondition, $P = N \geq 0$
- Nous voudrions prouver les POs:
  1. Etablir *Inv*: **(1)** $P \implies Inv$, autrement dit, prouver

$$\{P\}\ j := 0; s := 0;\ \{Inv\}$$

  2. Maintenir *Inv*: **(2)**

$$\{Inv \wedge B\}\ c\ \{Inv\} \iff$$
$$\{Inv \wedge B\}\ s := s + a[j]; j := j + 1\ \{Inv\}$$

  3. Conclure: **(3)**

$$Inv \wedge \neg B \implies Q$$

- Après la boucle, le résultat que nous voulons Q est

$$s = (\sum_i a[i] \mid 0 \leq i < N)$$

Correction partielle de *Prog*1: PO **(1)**

Déjà fait :) Nous avons montré

$$\{P\}\ j := 0; s := 0\ \{Inv\} \iff \{N \geq 0\}\ j := 0; s := 0\ \{0 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j)\}$$

$$W = WP(j := 0; s := 0,\ 0 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j))$$

$$= 0 \leq 0 \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < 0)$$

$$P \implies W\ ?$$

$$N \geq 0 \implies 0 \leq 0 \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < 0)\ ?$$

$$N \geq 0 \implies 0 \leq 0 \leq N \wedge\ 0 = (\sum_i a[i] \mid 0 \leq i < 0)$$

$$= N \geq 0 \implies (0 \leq N \wedge 0 = 0)\ \ ;\ 0 \leq 0 \quad \text{(true, somme vide est 0)}$$

$$= N \geq 0 \implies (0 \leq N)\ \ ;\ 0 = 0 \quad \text{(true, } P \wedge P \text{ est true)}$$

$$= true$$

Donc l'invariant est établi

Correction partielle de *Prog*1: PO **(2)**

$$\{0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \wedge j < N\} \ s := s + a[j]; j := j + 1$$

$$\{0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j)\}$$

$$\begin{aligned}
W &= WP(s := s + a[j]; j := j + 1, 0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j)) \\
&= WP(s := s + a[j], (0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j))[j \leftarrow j + 1]) \\
&= WP(s := s + a[j], (0 \leq j + 1 \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j + 1))) \\
&= (0 \leq j + 1 \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j + 1)[s \leftarrow s + a[j]] \\
&= (0 \leq j + 1 \leq N \wedge \ s + a[j] = (\sum_i a[i] \mid 0 \leq i < j + 1)
\end{aligned}$$

Correction partielle de *Prog*1: PO **(2)** cond

$$0 \leq j < N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j) \implies w\ ?$$

$$0 \leq j < N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j) \implies$$

$$(0 \leq j+1 \leq N \wedge\ s + a[j] = (\sum_i a[i] \mid 0 \leq i < j+1)$$

$$=0 \leq j < N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j) \implies$$

$$(-1 \leq j \leq N \wedge\ s + a[j] = (\sum_i a[i] \mid 0 \leq i < j+1)$$ (simplifier les bornes de j)

$$=0 \leq j < N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j) \implies$$

$$(-1 \leq j \leq N \wedge\ s + a[j] = (\sum_i a[i] \mid 0 \leq i < j) + a[j])$$ (séparer le dernier terme de somme)

$$=0 \leq j < N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j) \implies$$

$$(-1 \leq j \leq N \wedge\ s = (\sum_i a[i] \mid 0 \leq i < j))$$ (soustraire a[j] des deux côtés)

$=true$ (car $0 \leq j \implies -1 \leq j, j \in \mathbb{N}$)

Correction partielle de *Prog*1: PO **(3)**

$Inv \wedge \neg B \implies Q$?

$0 \leq j \leq N \wedge s = (\sum_i a[i] \mid 0 \leq i < j) \wedge j \geq N \implies s = (\sum_i a[i] \mid 0 \leq i < N)$

$= 0 \leq j \wedge j = N \wedge s = (\sum_i a[i] \mid 0 \leq i < j) \implies s = (\sum_i a[i] \mid 0 \leq i < N)$

(car $(j \leq N \wedge j \geq N) = (j = N)$)

$= 0 \leq N \wedge s = (\sum_i a[i] \mid 0 \leq i < N) \implies s = (\sum_i a[i] \mid 0 \leq i < N)$

(En remplaçant j *par* N puisque $(j = N)$)

$= true$

(car $X \wedge Y \implies Y$)

Correction totale de *Prog*1

variant $v = N - j$ pour la loupe du *Prog*1

```
j := 0;
s := 0;
while (j < N) do
    s := s + a[j];
    j := j + 1;
done
```

- Montrer que v est initialement positive:

$$(Inv \wedge B) \Longrightarrow v > 0$$

$$0 \leq j \leq N \wedge s = (\sum_i a[i] \mid 0 \leq i < j) \wedge j < N \Longrightarrow N - j > 0 \quad (1)$$

- Montrer que v est décroissante:

$$\{Inv \wedge B \wedge v = V\} c \{v < V \wedge v \geq 0\}$$

$$\{0 \leq j \leq N \wedge s = (\sum_i a[i] \mid 0 \leq i < j) \wedge j < N \wedge N - j = V\}$$

$$s := s + a[j]; j := j + 1$$

$$\{N - j < V \wedge N - j \geq 0\} \quad (2)$$

Terminaison de *Prog*1: PO 1

$$0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \ \wedge \ j < N \implies N - j > 0$$

$$= 0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \ \wedge \ j < N \implies N > j$$

(ajouter j des deux cotés)

$$= true$$

(car $(N > j) = (j < N), X \wedge Y \implies X$)

Terminaison de *Prog*1: PO 2

$$W = WP(s := s + a[j]; j := j + 1, N - j < V \wedge N - j \geq 0)$$
$$= WP(s := s + a[j], WP(j := j + 1, N - j < V \wedge N - j \geq 0))$$
$$= WP(s := s + a[j], (N - j < V \wedge N - j \geq 0)[j \leftarrow j + 1])$$
$$= WP(s := s + a[j], (N - (j + 1) < V \wedge N - (j + 1) \geq 0))$$
$$= (N - (j + 1) < V \wedge N - (j + 1) \geq 0)$$

$$0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \ \wedge \ j < N \wedge N - j = V \implies$$
$$(N - (j + 1) < V \wedge N - (j + 1) \geq 0) \ ?$$

Supposons vrai $0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \ \wedge \ j < N \wedge N - j = V$

Donc $N - j = V$ vrai Or $N - j = V \implies N - j - 1 < V$ vrai

Et $j < N \implies N - j > 0 \implies N - j - 1 > -1 \implies N - j - 1 \geq 0$ vrai

$$0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \ \wedge \ j < N \wedge N - j = V \implies$$
$$(N - j - 1 < V \wedge N - j - 1 > 1)$$

$$0 \leq j \leq N \wedge \ s = (\sum_i a[i] \mid 0 \leq i < j) \ \wedge \ j < N \wedge N - j = V \implies$$
$$(N - (j + 1) < V \wedge N - (j + 1) > 0)$$

Correction totale de $Prog1$

$Prog1$ est totalement correct, on a établi donc

$$[N \geq 0]\ Prog1\ [s = (\sum_{i} a[i] \mid 0 \leq i < N)]$$

Frama-C et WP

- Bien comprendre ce que la fonction **doit garantir**
- Identifier :
  - les **préconditions** (accès mémoire, validité, bornes...),
  - les **postconditions** (retour, effets sur pointeurs...)
- Écrire ces propriétés en langage logique via ACSL
- Laisser WP vérifier la logique de ces conditions automatiquement

- Frama-C calcule $wp(C, Q)$ à partir des contrats spécifiés en ACSL
- Il génère une ou plusieurs **conditions d'obligation de preuve** (VCs).
- Ces VCs sont ensuite envoyées à des **solveurs SMT** comme Alt-Ergo, Z3, ou Coq.

Avantage

Permet une vérification entièrement automatisée de la conformité du code à sa spécification.

Frama-C: Position dans le code et tableaux

- Pour parler de l'état d'une variable à un moment précis :
  - $\backslash old(x)$: valeur de x avant l'appel
  - $\backslash at(x, Label)$: valeur de x à un point nommé du code
- La validité des accès mémoire est vérifiée avec $\backslash valid$

```
/*@
  requires x >= 0;
  ensures \result == \old(x) + 1;
*/
int f(int x) {
  return x + 1;
}

/*@ requires \valid(t + (0..n-1)) && n > 0;
    ensures \result == t[0];
 */
int find_first(int* t, int n) {
  return t[0];
}
```

Frama-C: Exemple : somme d'entiers

```
/*@ requires n >= 0;
    ensures \result == (\sum integer i; 0 <= i < n; i);   */
int sum_n(int n) {
  int i = 0;
  int s = 0;
  /*@ loop invariant 0 <= i <= n;
      loop invariant s == (\sum integer j; 0 <= j < i; j);
      loop assigns i, s;
      loop variant n - i;      */
  while (i < n) {
    s += i;
    i++;
  }
  return s;
}
```

loop assigns:

- ↪ directive ACSL obligatoire pour la vérification WP
- ↪ Spécifie les variables/modifications permises dans la boucle
- ↪ Empêche la modification implicite d'autres variables
- ↪ Utile pour :
  - ↪ détecter des effets de bord cachés
  - ↪ s'assurer qu'aucune autre mémoire ne change